2024 Homework 1

Static CMOS, Data Selection, and XNOR

Introduction to VLSI Design / EE3032

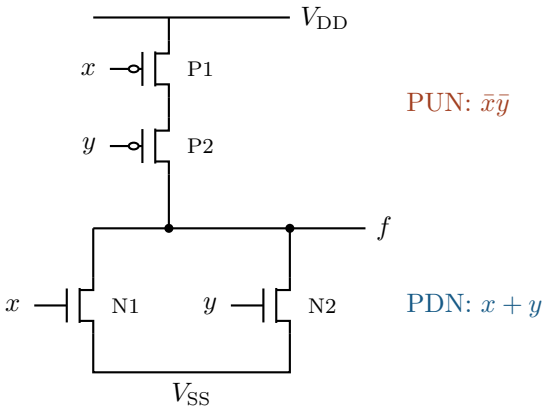
Method and notation. The solutions are derived from the original assignment, then compared with two user-supplied handwritten references. Here + means OR, juxtaposition means AND, and an overbar means NOT. Body terminals are omitted: pMOS bodies connect to V_{DD} and nMOS bodies to $V_{SS} = 0$. A gate bubble denotes a pMOS, which turns on for a low gate voltage. Problem 1 provides only true input literals; neither solution in Problem 1 requires an input inverter.

1(a) Absorption reduces the function to a NOR gate

Reading the original expression. The overbar covers the entire expression $x+y+wyz$. Therefore,

$$\begin{aligned} f(w,x,y,z) &= \overline{x+y+wyz} \\ &= \overline{x+y(1+wz)} = \boxed{\overline{x+y}}. \end{aligned}$$

Since $y + wyz = y$, the output is independent of w, z. The pull-down network (PDN) contains two parallel nMOS devices controlled by x, y. Its dual pull-up network (PUN) contains two pMOS devices in series.



x	y	w, z	PDN on	f
0	0	any	0	1
0	1	any	1	0
1	0	any	1	0
1	1	any	1	0

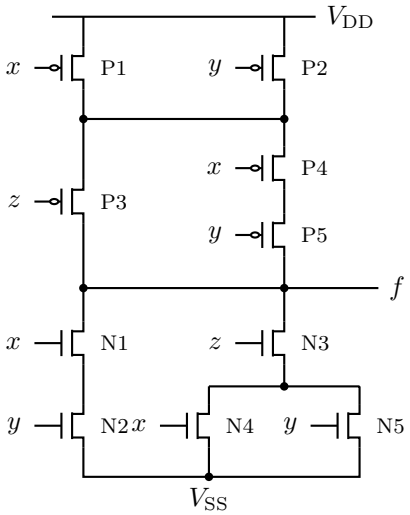
Selected implementation: a 4T NOR. Direct implementation of the unsimplified expression uses five nMOS and five pMOS devices. The wyz branch is redundant; eliminating it preserves the function and saves six transistors.

1(b) Three-input minority function: 10T static CMOS

The specified function is $f = \bar{x}\bar{y} + \bar{x}\bar{z} + \bar{y}\bar{z}$. Complementing the function gives a PDN expression using only true input literals:

$$\begin{aligned}\bar{f} &= (x+y)(x+z)(y+z) = xy + xz + yz \\ &= xy + z(x+y), \\ f &= \overline{xy + z(x+y)} = (\bar{x}+\bar{y})(\bar{z}+\bar{x}\bar{y}).\end{aligned}$$

The left PDN branch has x in series with y; the right branch has z in series with $(x \parallel y)$. The dual PUN has $(x \parallel y)$ in series with $[z \parallel (x \text{ in series with } y)]$. All gates use true input literals.



PUN: 5 pMOS

PDN: 5 nMOS

x	y	z	$xy + z(x+y)$	f	PUN on
0	0	0	0	1	1
0	0	1	0	1	1
0	1	0	0	1	1
0	1	1	1	0	0
1	0	0	0	1	1
1	0	1	1	0	0
1	1	0	1	0	0
1	1	1	1	0	0

Total: 10T, with no input inverters. The output is 1 exactly when at most one input is 1. Both references use valid 12T networks; factoring the function saves two transistors.

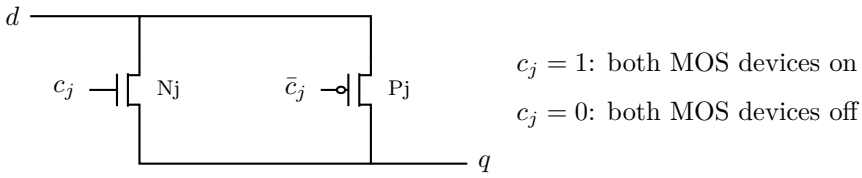
2. Four-way cyclic data selection using CMOS TGs

Each specified control word has exactly one $c_j = 1$. The design therefore assumes **one-hot control**. Four transmission gates (TGs) select one data input for each output:

$$z_3 = c_3a_3 + c_2a_2 + c_1a_1 + c_0a_0,$$
$$z_2 = c_3a_2 + c_2a_1 + c_1a_0 + c_0a_3,$$
$$z_1 = c_3a_1 + c_2a_0 + c_1a_3 + c_0a_2,$$
$$z_0 = c_3a_0 + c_2a_3 + c_1a_2 + c_0a_1.$$

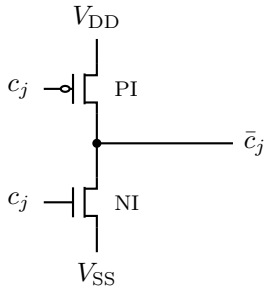
Equivalently, $z_i \leftarrow a_{(i+j-3) \bmod 4}$ when $c_j = 1$. The next page shows all 16 TG data connections. The four complementary control pairs $c_j, \bar{c}_j$ are shared across the circuit.

Transistor-level TG and complementary control



The two MOS devices are **in parallel between the same data terminals**. Unlike an nMOS-only pass gate, a CMOS TG passes both high and low logic levels without a threshold-drop limitation. Its data path is bidirectional: the upper and lower terminals in this drawing are data nodes, not supply rails. The bodies still connect to V_{SS} and V_{DD}, respectively.

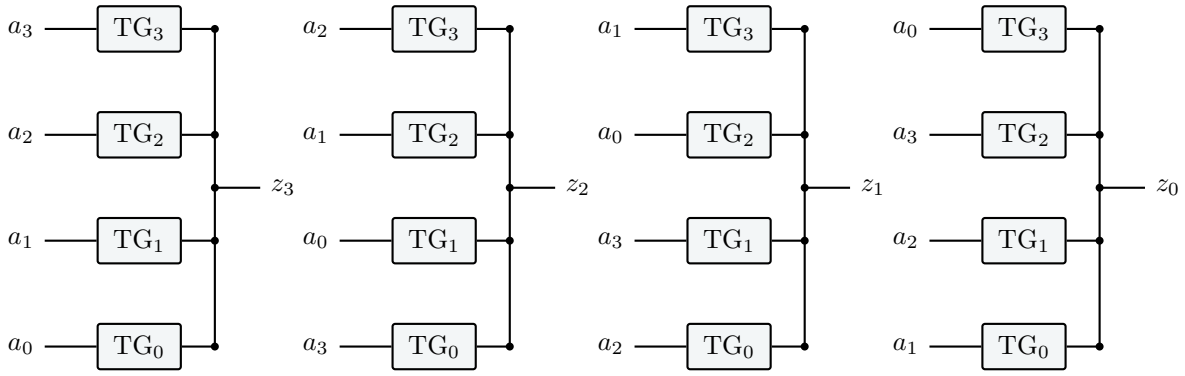
Shared control inverters (four instances, $j = 0, 1, 2, 3$)



Device count: 16 TGs × 2 + 4 inverters × 2 = 40T. No output buffers are included. Only the four one-hot words are specified. Zero selection leaves outputs floating; multiple selections can connect different data sources together. The controller must maintain one-hot control and avoid overlap during switching.

2. Complete TG connections and verification (continued)

Each TG_j expands to the two MOS devices on the preceding page: its nMOS gate connects to c_j and its pMOS gate to $\bar{c}_j$. All instances with the same control index share one inverter. Different output buses are not connected to each other.



c_3	c_2	c_1	c_0	Enabled TGs	z_3	z_2	z_1	z_0
1	0	0	0	All TG_3	a_3	a_2	a_1	a_0
0	1	0	0	All TG_2	a_2	a_1	a_0	a_3
0	0	1	0	All TG_1	a_1	a_0	a_3	a_2
0	0	0	1	All TG_0	a_0	a_3	a_2	a_1

Example control word. Set $c_2 = 1$ and all other controls to 0. Only TG_2 is enabled at each output. Tracing the connections gives

$$(z_3, z_2, z_1, z_0) = (a_2, a_1, a_0, a_3).$$

The other three control words follow identically. The supplied Python checker enumerates all four one-hot choices and all 16 data words, for $4 \times 16 = 64$ cases. It checks the expanded nMOS/pMOS connectivity as well as the equations.

If all control words must have defined outputs

The 12 unspecified control words are outside this one-hot TG design's operating condition. If the specification instead requires the AND-OR equations to hold for arbitrary $c_0, \ldots, c_3$, use four NAND2 gates producing $\overline{c_j a_k}$ and one NAND4 per output. This requires $4 \times 4 + 8 = 24\mathrm{T}$ per output, or 96T in total. It defines OR behavior for non-one-hot controls, at higher device cost than the selector needed for the original four specified words.

3(a) Logic function of the original pass-transistor circuit

The upper nMOS is controlled by A and passes B; the lower nMOS is controlled by $\bar{A}$ and passes $\bar{B}$. Hence $F = B$ for $A = 1$, and $F = \bar{B}$ for $A = 0$:

$$\boxed{F(A,B) = AB + \bar{A}\bar{B} = \overline{A \oplus B}} \qquad \text{(XNOR)}.$$

A	B	Enabled branch	Passed value	F	$\bar{F}$
0	0	Lower	$\bar{B} = 1$	1	0
0	1	Lower	$\bar{B} = 0$	0	1
1	0	Upper	$B = 0$	0	1
1	1	Upper	$B = 1$	1	0

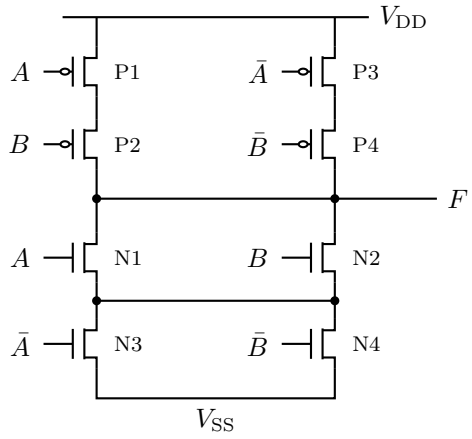
This logic analysis treats the original devices as ideal switches. An nMOS-only path can lose a threshold voltage when passing a high level; the original circuit should not be described as a full-swing static CMOS output stage.

3(b) Complementary static CMOS XNOR

For a static CMOS implementation, first write

$$\bar{F} = A\bar{B} + \bar{A}B = (A + B)(\bar{A} + \bar{B}).$$

The PDN is $(A \parallel B)$ in series with $(\bar{A} \parallel \bar{B})$. Its dual PUN has two parallel branches: A in series with B, and $\bar{A}$ in series with $\bar{B}$.



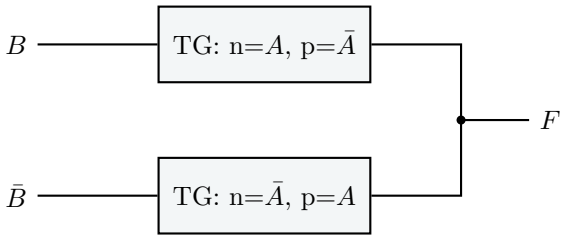
PUN: $AB + \bar{A}\bar{B}$

PDN: $(A + B)(\bar{A} + \bar{B})$

8T core, or 12T including complementary-input generation. If $\bar{A}, \bar{B}$ are unavailable, add two 2T inverters of the form shown in Problem 2, with inputs A, B, respectively. For every binary input word, exactly one of the PUN and PDN conducts.

3(b) Optional 8T transmission-gate implementation

A two-to-one CMOS TG selector implements the same function:



Each TG contains the two parallel MOS devices shown in Problem 2. Two TGs use 4T, and the two inverters generating $\bar{A}, \bar{B}$ use another 4T, for 8T total. Only the upper path conducts for $A = 1$; only the lower path conducts for $A = 0$. This mutually exclusive selection prevents B and $\bar{B}$ from being connected together. The primary answer remains the preceding static CMOS circuit because it explicitly demonstrates complementary PUN/PDN construction.

Sources and method of comparison

Original assignment: `homework1.pdf`, EE3032 Introduction to VLSI Design, Fall 2024, one page. The following pre-existing handwritten documents were supplied by the user. No author or license was identified. Their handwritten content is not presented as newly authored work here.

- **Reference A:** a user-supplied four-page HW1 correction/reference, including a cover. All solution content on pages 2–4 was reviewed.
- **Reference B:** a three-page student HW1 solution. All pages were reviewed.

The repository's source inventory records the original local-file provenance.

Reference review and selection rationale

Problem	Reference review	Selected solution
1(a)	Both retain the unsimplified 10T network. A labels the redundant branch with $\bar{z}$, contrary to the true-literal restriction; absorption makes this branch irrelevant to the output.	Read the original overbar correctly and simplify to $\overline{x+y}$: a 4T NOR.
1(b)	Both use valid 12T networks with PDN $(x+y)(x+z)(y+z)$.	Preserve the minority function, factor the PDN as $xy+z(x+y)$, and use 10T.
2	Both have the correct cyclic selection wiring. B explicitly shows inverted controls; A uses more abbreviated notation.	Retain the valid wiring; expand the TG gate controls and shared inverters. State the one-hot condition and the 40T count.
3	Both identify XNOR. A's complementary control bars can be distinguished, but the upper MOS-type symbol is unclear. B's static CMOS structure is valid.	Use the independently derived static CMOS circuit, consistent with B. Redraw the complete TG alternative with explicit n/p controls; do not infer a wiring error from A's ambiguous symbol.

Reproducible verification

`verify_logic.py` enumerates the input words and builds conducting graphs from the MOS terminals in the diagrams: 16 cases for 1(a), 8 for 1(b), 64 specified cases for 2, and 4 for each implementation of 3. It checks that static CMOS has neither floating outputs nor rail shorts, and that each TG output reaches exactly the selected data source. Inverter checks and unspecified-control examples bring the total to 100 cases. These are ideal binary-switch checks, not HSPICE process, delay, switching-transient, or analog-voltage simulations.